cl-agent: A Continual-Learning Substrate for Coding Agents

Episode Capture, Replay, and Rule-Based Distillation for Cross-Session Improvement Without Fine-Tuning

Datt Goswami
Independent Research
dattgoswami@gmail.com

April 2026

Abstract

Large language model (LLM)-based coding agents are increasingly capable within a single session, yet they typically remain amnesic across sessions. Each new run begins without explicit access to prior attempts, failed tests, successful repair patterns, or repository-specific conventions. This paper introduces **cl-agent**, a *continual-learning substrate* for coding agents that accumulates durable, inspectable knowledge across repeated sessions without modifying model weights. Drawing on the episodic memory and experience-replay traditions in continual learning (7; 20), cl-agent defines four responsibilities: *episode capture*, *replay*, *distillation*, and *evaluation*. Episodes are stored as normalized, append-only JSONL records; a heuristic replay buffer surfaces relevant failures and successes before each new run; a rule-based distillation pipeline produces inspectable Markdown artifacts (SKILLS.MD, DREAMS.MD, PROGRAM.MD); and an evaluation layer separates operational metrics from research claims requiring controlled benchmarks. The substrate assumes no access to model internals—explicit context injection through `developer_instructions` is the only coupling point. We describe the architecture, relate each design decision to established continual-learning theory, and articulate a narrow, falsifiable v1 claim: a thin substrate built from episode recording, replay, and rule-based distillation can measurably improve coding-agent performance across repeated tasks in a narrow domain without fine-tuning the underlying model. We situate cl-agent within the lifelong learning literature (17; 18), discuss its relationship to catastrophic forgetting (1; 2; 3), and outline an evaluation protocol that separates operational utility from research-grade transfer claims.

1 Introduction

LLM-based coding agents have advanced rapidly in their ability to write, debug, refactor, and test code within a single session (24; 25; 26). State-of-the-art systems can autonomously execute complex engineering tasks when given a well-specified prompt and sufficient context. Yet each session begins from scratch. An agent that spent two hours yesterday tracing why a FastAPI endpoint triggered a Pydantic validation error will make the same diagnostic journey today—there is no mechanism by which successful patterns propagate forward as structured, inspectable knowledge.

This is not a failure of the underlying model; it is a failure of the *interface between the model and its environment*. Model weights do not change between sessions (fine-tuning is expensive, data-scarce, and risks catastrophic forgetting on other tasks (3)). What is missing is a lightweight layer that observes what the agent does, extracts what worked, and injects that knowledge explicitly before the next run. This is precisely the role that episodic memory plays in biological cognition (20) and that experience replay plays in artificial continual learning (6; 7).

We introduce **cl-agent**, a Codex-first continual-learning substrate for coding agents. Its design philosophy is deliberately conservative:

- It does not fine-tune the underlying model.
- It does not wrap, replace, or compete with the agent.
- It does not depend on vector databases, embeddings, or ML inference at distillation time.
- It persists knowledge as plain Markdown files that any agent can read and any human can inspect or edit.

The system is grounded in four responsibilities borrowed from the continual learning literature and adapted to the coding-agent setting: *capture* (recording what happened with maximum fidelity), *replay* (retrieving relevant past experience before a new run), *distillation* (extracting durable patterns from the episode history), and *evaluation* (measuring whether cross-session improvement is real and attributable).

1.1 Contributions

1. A normalized episode schema and append-only recorder for coding-agent runs that preserve raw observable outcomes before any scalar reward compression.
2. A domain-heuristic replay buffer that surfaces recent failures and same-domain successes without embeddings, proving the substrate claim at minimum infrastructure cost.
3. A rule-based distillation pipeline producing three inspectable, human-editable artifacts: SKILLS.MD, DREAMS.MD, and PROGRAM.MD.
4. A two-mode operating protocol (*baseline* vs. *integrated*) that separates substrate-driven improvement from the agent's own native memory, enabling cleaner attribution.
5. An evaluation framework that distinguishes operational metrics (computable from any run) from research metrics (forward and backward transfer) that are only valid under a controlled benchmark protocol.

2 Background and Related Work

2.1 Catastrophic Forgetting and Continual Learning

The canonical challenge in continual learning is *catastrophic forgetting*: when a neural network is trained sequentially on new tasks, it catastrophically overwrites representations useful for prior tasks (1; 2). This observation, originating in connectionist cognitive science, motivates the entire continual-learning research programme.

Three broad families of methods have been developed in response.

Regularization-based methods. These approaches add penalty terms to the training loss that discourage changes to parameters deemed important for prior tasks. Elastic Weight Consolidation (EWC; 3) uses the Fisher information matrix to identify and protect task-critical weights. Synaptic Intelligence (SI; 4) accumulates per-parameter importance online during training. Learning without Forgetting (LwF; 5) uses knowledge distillation to preserve prior task outputs without storing exemplars.

Replay-based methods. These approaches maintain a buffer of prior examples or generated proxies and interleave them with new training data. Gradient Episodic Memory (GEM; 6) constrains gradient updates to avoid increasing loss on stored exemplars. A-GEM (8) scales GEM to larger settings. Experience replay (7) revisits raw stored trajectories. iCaRL (9) combines replay with prototype-based classification. Generative replay (10) uses a companion generative model to synthesize past examples on demand.

Architecture-based methods. These approaches prevent interference through structural partitioning or network expansion. Progressive Neural Networks (13) allocate fresh columns per task. PackNet (14) iteratively prunes and reuses parameters without explicit task boundaries.

cl-agent occupies none of these positions because it does not modify model parameters. It is instead a *substrate-level* approach: it captures, replays, and distills experience to compose better *context* for a frozen model at inference time. This places it closer to the lifelong machine learning framework of **(author?)** (17) and the case-based reasoning tradition (32) than to gradient-level continual learning.

2.2 Episodic Memory and Experience Replay

The episodic memory concept—memory for specific events bound in time and context (20)—is the most direct theoretical antecedent of cl-agent's episode model. Each coding-agent run is treated as an episodic event: a bounded temporal experience with a start, a sequence of observable actions, and an outcome. The EpisodeRecorder is an externalized episodic memory system.

Experience replay, originally proposed to stabilize learning in reinforcement agents (22), exploits stored experience to break the non-stationarity of sequential learning. cl-agent's replay buffer implements a domain-heuristic variant: recent failures are surfaced as warnings; recent successes in the same task domain are surfaced as positive guidance. This is functionally analogous to prioritized experience replay (12) but operates over full episodes rather than individual transitions, and at the prompt-composition level rather than the gradient level.

2.3 Lifelong Learning for Agents

The lifelong learning paradigm (18) envisions systems that accumulate knowledge indefinitely and transfer it usefully to new tasks. Ring's thesis (19) established the formal basis for continual learning in reinforcement learning systems. The forward transfer (FWT) and backward transfer (BWT) metrics formalised by **(author?)** (6) and refined by **(author?)** (8) provide a quantitative framework for measuring whether learning on new tasks helps or hurts performance on old ones.

cl-agent adopts this evaluation vocabulary but guards BWT/FWT metrics behind an explicit benchmark protocol requirement (Section 8), avoiding the common failure mode of reporting transfer statistics from uncontrolled ad hoc runs (16).

2.4 Memory Systems for LLM Agents

Recent work on LLM-based agents has begun to address the cross-session memory gap. Generative Agents (27) introduce a memory stream architecture combining recency, importance, and relevance weighting to retrieve relevant memories at interaction time. Reflexion (28) uses verbal reinforcement—natural language summaries of past failures—to improve future attempts on the same task. Voyager (29) accumulates an explicit skill library for Minecraft tasks, growing a reusable procedural knowledge base through interaction.

These systems demonstrate that cross-session learning is achievable with frozen models. cl-agent differs in three key respects: (1) it targets *repeated coding sessions* rather than interactive dialogue or open-ended game play; (2) its distilled knowledge is stored in structured, human-editable Markdown rather than opaque vector embeddings; and (3) it explicitly separates its learning signal from the agent's native memory pipeline to enable controlled empirical attribution.

2.5 Coding-Agent Benchmarks

HumanEval (24) and SWE-bench (25) established rigorous evaluation protocols for coding agents. SWE-bench in particular tests agents on real GitHub issues, providing a realistic distribution of tasks with ground-truth test suites. SWE-agent (26) demonstrated that agent–computer interface design has a first-order effect on task

success rate. cl-agent's evaluation framework (Section 8) is designed to be compatible with SWE-bench-style task suites while adding the temporal dimension—repeated exposure to a fixed domain—necessary to measure continual improvement.

3 The cl-agent Framework

3.1 Design Principles

Five principles constrain every design decision in the framework.

1. **Observe before abstracting.** Raw observable outcomes (command exit codes, files changed, test results) are always preserved before any derived scalar is computed.
2. **Deterministic before learned.** Version 1 distillation is fully rule-based. LLM-based synthesis is explicitly deferred until the deterministic pipeline has been validated under a real benchmark.
3. **File-first persistence.** All artifacts are plain text. Any agent that can read a file benefits from them without additional infrastructure.
4. **Explicit injection.** Context derived from prior episodes is *never* assumed to reach the agent automatically. The adapter must compose and inject it explicitly via `developer_instructions`.
5. **Mode separation.** Baseline experiments must be free from the agent's native memory effects. The two-mode protocol (Section 7.3) enforces this.

3.2 Architecture Overview

The adapter and the CL layer are separated so that supporting a new agent surface requires only a new adapter—the core substrate is unchanged. The adapter's sole responsibilities are: (1) assembling the run context from prior artifacts, (2) driving the agent's run lifecycle, (3) collecting structured outputs, and (4) mapping them to the normalized episode schema.

3.3 The Four Pillars

Pillar 1: Episode Capture. The system records what the agent did with maximum fidelity. Every run produces one episode: a bounded temporal record with structured events and an outcome. Raw observable signals are always preserved; derived quantities are computed later by evaluators.

Pillar 2: Replay. Before each new run, the replay buffer is queried for relevant prior experience: recent failures serve as warnings, and recent successes in the same task domain serve as positive context. This mirrors the prioritized replay intuition of **(author?)** (12) at the episode level.

Pillar 3: Distillation. After each run (or batch of runs), a deterministic distillation pipeline extracts structured artifacts. Repeated successes in a domain become skill candidates. Repeated failures become warnings. Session summaries become DREAMS.MD. The next-run injection context becomes PROGRAM.MD.

Pillar 4: Evaluation. A structured evaluation layer computes operational metrics on every run and guards research metrics (BWT, FWT) behind an explicit benchmark protocol. No cross-session learning claim is made without the controlled conditions those metrics require.

```
+-----------------------------------------------------------------+
|  Agent Surface                                                  |
|  Codex (v1)  ·  Hermes (planned)  ·  others later              |
+---------------------------+-------------------------------------+
                            |
                            v
+-----------------------------------------------------------------+
|  Thin Adapter   (adapters/codex/)                               |
|  context_builder.py  ->  assembles run context explicitly       |
|  sdk_runner.py       ->  drives Codex thread lifecycle          |
|  item_mapper.py      ->  maps ThreadItems -> episode events     |
+---------------------------+-------------------------------------+
                            |
                            v
+-----------------------------------------------------------------+
|  CL Layer   (cl-layer/)                                         |
|  episode/   -- normalized schema + JSONL recorder               |
|  replay/    -- domain-heuristic buffer, no embeddings           |
|  distill/   -- rule-based skills, dreams, program writers       |
|  eval/      -- operational + research metrics  (Phase 4)        |
|  mcp/       -- MCP server interface            (Phase 6)        |
+---------------------------+-------------------------------------+
                            |
                            v
+-----------------------------------------------------------------+
|  Persistent Artifacts                                           |
|  episodes.jsonl   SKILLS.md   DREAMS.md   PROGRAM.md            |
+-----------------------------------------------------------------+
```

Figure 1: cl-agent architecture. The adapter is the only agent-surface-specific component. The CL layer and all persistent artifacts are surface-agnostic.

4 Episode Model

The episode model is the substrate's primary data structure. Its design follows the principle that raw outcomes are always more valuable than compressed scalars (31).

4.1 Schema

Listing 1: Episode schema (simplified).

```python
@dataclass
class EpisodeEvent:
    kind: Literal["command_execution", "file_change",
                  "mcp_tool_call", "agent_message",
                  "evaluation_result"]
    timestamp: datetime
    payload: dict          # raw structured data from the agent

@dataclass
class EpisodeOutcome:
    status: Literal["completed", "partial", "failed", "escalated"]
    tests_passed: bool | None     # None until evaluator fills it
    verification_summary: str | None
```

```python
    escalation_reason: str | None
    files_touched: list[str]
    final_response: str | None

@dataclass
class Episode:
    episode_id: str
    run_id: str
    thread_id: str | None          # agent-surface thread identifier
    task_id: str
    task_description: str
    task_domain: str
    agent_surface: str             # "codex", "hermes", ...
    mode: Literal["baseline", "integrated"]
    started_at: datetime
    ended_at: datetime
    events: list[EpisodeEvent]
    outcome: EpisodeOutcome
    reward: float | None           # None at record time; evaluator-derived
```

The critical design decision is that `reward` is `None` at record time. This preserves the raw evidence from which multiple reward framings can be computed later, avoiding the irreversibility of premature scalar compression. This parallels the advice of **(author?)** (30) to keep environment signals separate from learned value estimates, and the more direct observation of **(author?)** (31) that reward definition is often a non-trivial research question in real-world settings.

4.2 Serialization

Episodes are serialized to append-only JSONL. Each line is a complete, self-contained JSON object. The format is intentionally simple: it requires no schema migrations, can be read with any JSON parser, and supports streaming aggregation with standard tooling such as `jq` or DuckDB. The recorder is resilient to malformed lines—corrupted entries are skipped without disrupting the session.

5 Replay Buffer

The replay buffer implements a heuristic retrieval policy without embeddings or vector similarity. The policy reflects two observations from the continual-learning literature.

1. **Recent failures are the most valuable warnings.** A failure from the last session is more likely to recur than one from three months ago. Recency weighting for negative experience mirrors the priority given to high-TD-error transitions in **(author?)** (12).
2. **Domain similarity is a reliable proxy for task relatedness.** In the absence of semantic embeddings, task domain (e.g., `fastapi`, `rust-async`) is a strong structural proxy for skill transferability. This reflects the transfer learning intuition that similarity in input distribution predicts positive transfer (23).

5.1 Query Policy

Let $\mathcal{E}_{\text{fail}}$ denote prior episodes whose outcome status is `failed`, `partial`, or `escalated`, and let $\mathcal{E}_{\text{success}}$ denote episodes whose status is `completed`. Given a task domain d and operating mode m, the replay set is

$$\mathcal{R}(d, m) = \mathcal{F}_m(\text{top-}\, k_f(\mathcal{E}_{\text{fail}}) \cup \text{top-}\, k_s(\mathcal{E}_{\text{success}} \cap \mathcal{D}_d)), \tag{1}$$

where $\mathcal{D}_d$ is the domain filter, and $\mathcal{F}_m$ filters by operating mode. Episodes within each subset are sorted by `ended_at` (most recent first). The caps k_f and k_s are configurable (default 5 each).

This policy is *deterministic* for a fixed episode store and query parameters, making it fully testable without a live agent. The 22 unit tests covering episode serialization and replay ordering all run without network access.

6 Distillation Layer

The distillation layer transforms raw episode histories into structured, human-readable artifacts that persist across sessions. Version 1 distillation is entirely rule-based: no LLM inference is required. This choice reflects the principle that a deterministic, auditable system must be demonstrated to work before an opaque learned one is introduced.

6.1 Skill Distillation

A domain qualifies for a skill candidate when it accumulates at least $\tau = 2$ completed episodes (configurable):

$$\text{SkillCandidate}(d) \iff |\{e \in \mathcal{E} : e.\text{domain} = d \wedge e.\text{status} = \texttt{completed}\}| \geq \tau. \tag{2}$$

Each candidate preserves its `evidence_episode_ids`, ensuring every skill is traceable to its source episodes. This addressability is analogous to case indexing in case-based reasoning (32): skills are not free-floating assertions but evidence-backed claims that can be challenged and retracted.

6.2 Warning Distillation

Domains with repeated failures produce explicit warnings:

$$\text{Warning}(d) \iff |\{e \in \mathcal{E} : e.\text{domain} = d \wedge e.\text{status} \in \{\texttt{failed}, \texttt{partial}, \texttt{escalated}\}\}| \geq \tau_w. \tag{3}$$

Warnings are appended to PROGRAM.MD and surfaced in the next run's `developer_instructions`.

6.3 Persistent Artifacts

Table 1: Persistent artifacts produced by the distillation layer.

Artifact	Format	Updated	Purpose
`episodes.jsonl`	JSONL	Per run	Append-only raw evidence store
SKILLS.MD	Markdown	Post-run	Evidence-backed reusable patterns
DREAMS.MD	Markdown	Post-session	Session summary: outcomes, recurrences, replay targets
PROGRAM.MD	Markdown	Post-session	Next-run injection context: domain, patterns, warnings

All artifacts are designed to be read by humans and diffed in version control. A skill entry that appears incorrect can be edited or deleted directly; the episode store remains the authoritative ground truth.

7 Agent Integration

7.1 SDK-Based Integration

The initial design assumed a CLI-based integration path. Inspection of the local Codex SDK revealed that the Python `codex_app_server` package provides a structured interface that makes CLI parsing unnecessary and brittle:

Listing 2: SDK-based integration (simplified).

```python
from codex_app_server import Codex

with Codex() as codex:
    thread = codex.thread_start(
        cwd="/path/to/repo",
        developer_instructions=context.developer_instructions,
        ephemeral=True,          # baseline mode
        model="o4-mini",
    )
    result = thread.run(task_prompt)
    # result.items: list[ThreadItem] -- typed, no log parsing
```

`result.items` is a `list[ThreadItem]`, a Pydantic discriminated union. The adapter maps these directly by `item.type` using duck-typing, ensuring forward compatibility as the SDK evolves.

7.2 Item Mapping

Table 2: Codex `ThreadItem` types and their episode event mappings.

Codex `type`	Fields used	`EpisodeEvent.kind`
`commandExecution`	`command`, `exit_code`, `aggregated_output`, `duration_ms`	`command_execution`
`fileChange`	`changes[*].path`, `status`	`file_change`
`mcpToolCall`	`tool`, `server`, `status`, `duration_ms`	`mcp_tool_call`
`agentMessage`	`text`, `phase`	`agent_message`
all others	—	skipped

7.3 Operating Modes

Two modes are defined to separate experimental conditions:

Table 3: Operating mode semantics.

Mode	Codex thread	Context injection	Use for
`baseline`	`ephemeral=True` (native memory minimized)	Off — substrate is the only cross-session signal	Controlled experiments; clean attribution
`integrated`	Persistent (native memory active)	On — PROGRAM.MD + SKILLS.MD injected as `developer_instructions`	Daily practical use

The mode distinction is load-bearing for research integrity. Any improvement observed under `baseline` conditions cannot be attributed to Codex's native memory pipeline—the substrate is the only deliberate

cross-session mechanism. This separation is analogous to the ablation discipline required when evaluating replay-based continual learning methods against regularization baselines (8).

8 Evaluation Framework

8.1 Operational Metrics

Operational metrics are computable from any run's episode record and require no benchmark protocol:

$$\text{Completion Rate} = \frac{|\{e : e.\text{status} = \texttt{completed}\}|}{|\mathcal{E}|}, \tag{4}$$

$$\text{Escalation Rate} = \frac{|\{e : e.\text{status} = \texttt{escalated}\}|}{|\mathcal{E}|}, \tag{5}$$

$$\text{Failure Recurrence} = \frac{|\{e : e.\text{task_id} \in \mathcal{F}_{\text{prior}}\}|}{|\mathcal{F}_{\text{prior}}|}. \tag{6}$$

Additional operational metrics include commands per successful task, files touched per task, runtime, and cost per solved task when token counts are available.

8.2 Research Metrics: Forward and Backward Transfer

BWT and FWT are adapted from **(author?)** (6). Let $a_{i,j}$ denote the performance (task success rate) on task j after the agent has been exposed to tasks up through task i, and let b_j denote the cold-start baseline on task j with no prior episodes:

$$\text{BWT} = \frac{1}{T-1} \sum_{i=1}^{T-1} (a_{T,i} - a_{i,i}), \tag{7}$$

$$\text{FWT} = \frac{1}{T-1} \sum_{i=2}^{T} (a_{i-1,i} - b_i). \tag{8}$$

These metrics are only valid under a controlled benchmark protocol. The protocol requires:

1. A curated task suite in a narrow, fixed domain (e.g., repeated FastAPI endpoint tasks).
2. A true cold-start baseline run (b_j) with `baseline` mode and no prior episodes.
3. A known and fixed task exposure order.
4. Separate measurement of each $a_{i,j}$ without contamination across conditions.

Presenting BWT/FWT from ad hoc, uncontrolled runs is explicitly prohibited by the framework design. This reflects the lesson from the continual learning literature that informal ablations routinely overstate transfer (16).

9 The Session Loop

The full interaction loop, from run initiation to artifact update, proceeds as follows.

Pre-run (integrated mode only):

- Read PROGRAM.MD and SKILLS.MD from `artifacts_dir`.
- Compose `developer_instructions` by concatenating available artifacts.
- Call `thread_start(..., developer_instructions=...)` with the composed context.

Run:

- Call `thread.run(task_prompt)`.
- Collect `result.items` (list of typed `ThreadItems`).
- Map items to `EpisodeEvent` objects via the item mapper.

Post-run:

- Write episode to `episodes.jsonl` (append-only).
- Run `distill/skills.py` → update SKILLS.MD.
- Run `distill/dreams.py` → update DREAMS.MD.
- Run `distill/program.py` → update PROGRAM.MD for the next run.
- Compute operational metrics (Phase 4).

This loop closes the continual-learning cycle: each run contributes to the episode store, which feeds distillation, which shapes the context of the next run. The loop is the product; the individual artifacts are intermediate representations.

10 Architectural Guardrails

Several design decisions are explicitly load-bearing and should not be reversed without benchmark evidence. Table 4 enumerates these constraints and their rationale.

11 Discussion

11.1 Relationship to Catastrophic Forgetting

cl-agent does not suffer from catastrophic forgetting in the classical parameter-overwriting sense because it does not update model weights. Instead, it faces a symmetric problem at the prompt level: an agent that reads only its most recently distilled context may lose access to patterns from earlier sessions if those patterns are not promoted into the current SKILLS.MD. We call this *representational forgetting at the prompt level*.

The skill promotion threshold ($\tau = 2$) and the append-only episode store address this: patterns must recur to be promoted, and the raw evidence is never discarded. A skill that is removed from SKILLS.MD can always be reconstructed from `episodes.jsonl`. This mirrors the spirit of complementary learning systems theory (21), where the hippocampus (episode store) preserves specific events while the neocortex (distilled artifacts) accumulates statistical regularities slowly.

11.2 Scalability of File-First Persistence

The flat-file approach is appropriate for version 1 because it eliminates operational overhead, is immediately inspectable, and is portable to any agent that can read text. At scale—thousands of episodes across dozens of domains—domain-filtered full-table scans over `episodes.jsonl` will become a bottleneck. DuckDB or a lightweight indexed store can be layered on as a backend without changing the substrate's external interface, since the replay buffer's contract specifies only `load_all()` and returns typed `Episode` objects.

Table 4: Architectural guardrails and their rationale.

Guardrail	Rationale
Structured `ThreadItem`s over log parsing	Log parsing is brittle and ties the substrate to a specific output format. Typed SDK models are versioned and stable.
Baseline mode uses ephemeral threads	Mixing Codex native memory into baseline experiments invalidates learning attribution.
Explicit context injection	Files placed in a working directory are not automatically visible to Codex. The adapter must pass them as `developer_instructions`.
`reward = None` at record time	Premature scalar compression discards evidence. Multiple reward framings must be computable from the same raw episode (31).
Rule-based distillation first	Deterministic rules make the learning loop testable and auditable. LLM synthesis adds opacity before the signal is proven.
No vector search in version 1	Domain keyword filtering is sufficient to test the substrate claim. Vector search adds infrastructure risk and retrieval unpredictability.
Narrow benchmark before broad claims	BWT/FWT are only meaningful under controlled exposure conditions (6).

11.3 LLM-Based Distillation as a Phase-2 Upgrade

Version 1 distillation produces coarse skill candidates: "domain d produced k completions." An LLM-based synthesis step could extract finer-grained patterns: specific function signatures that consistently succeeded, test fixtures that were reliably stable, and error classes that repeatedly caused failures. This is a direct analogue of the generative replay approach of **(author?)** (10), applied at the knowledge-artifact level rather than the data-generation level.

The correct time to introduce this upgrade is after the deterministic pipeline has been validated under a real benchmark. Adding LLM synthesis before that point makes it impossible to isolate whether any observed improvement comes from the substrate's structure or from the language model's summarization ability.

11.4 Multi-Agent Extension

The file-first artifact design is natively shareable: multiple agents reading the same SKILLS.MD share the same distilled knowledge base. A team of agents working on the same repository could converge on a shared episode store, enabling a form of federated lifelong learning (17) without any dedicated coordination infrastructure. This extension is explicitly deferred to later phases.

12 Implementation Roadmap

The v1 credibility milestone is Phase 5: a Markdown report showing that prior episodes in a controlled narrow-domain benchmark measurably helped subsequent tasks in the same domain. Until that report exists, improvement is a plausible hypothesis, not a demonstrated result. The design specifically avoids making empirical claims before this milestone.

Table 5: cl-agent build phases and current implementation status.

Phase	Name	Status	Key Deliverable
0	Architecture lock	**Done**	`spec.md`, `spec-validation.md`, `plan.md`
1	Core data plane	**Done**	`episode/`, `replay/`, 22 passing tests
2	Distillation	**Done**	`distill/skills.py`, `distill/dreams.py`, `distill/program.py`
3	Codex adapter	**Done**	`adapters/codex/` grounded in the local SDK
4	Evaluation layer	Planned	`cl_layer/eval/reward.py`, `cl_layer/eval/metrics.py`
5	Benchmark harness	Planned	Narrow domain suite, baseline vs. integrated comparison
6	MCP interface	Planned	`cl_layer/mcp/server.py`
7	Second surface	Planned	Hermes adapter, core substrate unchanged

13 Conclusion

We have presented cl-agent, a continual-learning substrate for coding agents that implements four pillars—episode capture, replay, distillation, and evaluation—without modifying the underlying model or replacing the agent. The system is grounded in established continual-learning theory (episodic memory, experience replay, knowledge distillation, forward/backward transfer metrics) while making pragmatic choices appropriate for the production coding-agent setting: file-first persistence, rule-based distillation, and a two-mode protocol that preserves experimental integrity.

Three phases are complete: the core data plane (22 passing tests, zero network dependencies), the rule-based distillation layer, and the Codex adapter grounded in the real SDK. Four phases remain: an evaluation layer, a narrow benchmark harness that will make the first credible BWT/FWT measurement possible, an MCP interface for portability, and a second agent surface that will demonstrate the substrate is not Codex-specific.

The central claim this work is positioned to support at v1 is:

> *A thin substrate built from episode recording, replay, rule-based skill distillation, and explicit context injection can improve coding-agent performance across repeated tasks in a narrow domain without fine-tuning the underlying model.*

That claim—and the controlled benchmark protocol designed to test it—represents the first step toward a rigorous empirical science of continual learning for software engineering agents.

Limitations. This paper is best interpreted as a systems position paper together with an implementation proposal. Strong empirical transfer claims should be deferred until the benchmark harness (Phase 5) and evaluation layer (Phase 4) are complete. The v1 distillation pipeline produces coarse skill representations that may miss fine-grained procedural knowledge. The flat-file design, while inspectable, will require a more efficient backend before scaling to thousands of episodes.

References

[1] McCloskey, M. and Cohen, N. J. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989. Academic Press. 10.1016/S0079-7421(08)60536-8

[2] French, R. M. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999. 10.1016/S1364-6613(99)01294-2

[3] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., and Hadsell, R.

Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 10.1073/pnas.1611835114

[4] Zenke, F., Poole, B., and Ganguli, S. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 3987–3995. PMLR, 2017.

[5] Li, Z. and Hoiem, D. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2017. 10.1109/TPAMI.2017.2773081

[6] Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. Curran Associates, 2017.

[7] Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T. P., and Wayne, G. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32. Curran Associates, 2019.

[8] Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Rebuffi, S.-A., Kolesnikov, A., Sperl, G., and Lampert, C. H. iCaRL: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2001–2010, 2017. 10.1109/CVPR.2017.587

[10] Shin, H., Lee, J. K., Kim, J., and Kim, J. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. Curran Associates, 2017.

[11] Riemer, M., Cases, I., Ajemian, R., Liu, M., Rish, I., Tu, Y., and Tesauro, G. Learning to learn without forgetting by maximizing transfer and minimizing interference. In *International Conference on Learning Representations (ICLR)*, 2019.

[12] Schaul, T., Quan, J., Antonoglou, I., and Silver, D. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[13] Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[14] Mallya, A. and Lazebnik, S. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7765–7773, 2018. 10.1109/CVPR.2018.00810

[15] Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., and Wermter, S. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019. 10.1016/j.neunet.2019.01.012

[16] De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., and Tuytelaars, T. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2022. 10.1109/TPAMI.2021.3057446

[17] Chen, Z. and Liu, B. *Lifelong Machine Learning*, 2nd edition. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2018. 10.2200/S00832ED1V01Y201802AIM037

[18] Thrun, S. and Mitchell, T. M. Lifelong robot learning. *Robotics and Autonomous Systems*, 15(1–2):25–46, 1995. 10.1016/0921-8890(95)00004-Y

[19] Ring, M. B. *Continual Learning in Reinforcement Learning Systems.* PhD thesis, University of Texas at Austin, 1994.

[20] Tulving, E. *Elements of Episodic Memory.* Oxford University Press, 1983.

[21] McClelland, J. L., McNaughton, B. L., and O'Reilly, R. C. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. 10.1037/0033-295X.102.3.419

[22] Lin, L.-J. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. 10.1007/BF00992699

[23] Pan, S. J. and Yang, Q. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010. 10.1109/TKDE.2009.191

[24] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[25] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[26] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[27] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th ACM Symposium on User Interface Software and Technology (UIST)*. ACM, 2023. 10.1145/3586183.3606763

[28] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[29] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research (TMLR)*, 2023.

[30] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[31] Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Paduraru, C., Gowal, S., and Hester, T. Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning*, 110:2419–2468, 2021. 10.1007/s10994-021-05961-4

[32] Kolodner, J. L. An introduction to case-based reasoning. *Artificial Intelligence Review*, 6(1):3–34, 1992. 10.1007/BF00155578